

Your solution is judged in layers. First, every team goes through automated API tests. Then, the shortlisted teams go through a manual review. The exact hidden test design, internal labels, and expected answers remain confidential.

Layer 1: The Seven Scoring Categories

#	Category	Weight	What it really measures	Simple explanation
1	Evidence Reasoning	35	Can the service solve the task using the supplied case data, identify the relevant evidence, and produce the right review outcome?	This is the most important category. The judge must reason about the evidence and match key points from the case data.
2	Safety & Escalation	20	Does the service avoid unsafe behaviour, protect sensitive information, and route uncertain or risky situations to human review?	Safety is a critical requirement. Unsafe responses, even when they look correct, are penalised.
3	API Contract & Schema	15	Does the response look exactly like the spec? Right fields, right types, right enum values, and right HTTP codes?	The judge checks the JSON shape. It cannot rely on reasoning about the content.
4	Performance & Reliability	10	Is it fast enough, stable under judging, and able to handle unexpected input without crashing?	Your API should be robust. The timeout is 30 seconds. It should handle unexpected input safely on a shared environment.
5	Response Quality	10	Is the generated text useful? Clear summary, practical next action,	Shortlisted teams are judged on whether their responses are actually useful and safe for users.

			professional customer reply?	
6	Deployment & Reproducibility	5	Can judges run or reach the service without asking the team for help?	A good so accessible endpoint c the Docker
7	Documentation	5	Does the README explain how it works, what AI was used, safety logic, and limitations?	Your REA judges un choices, s limitations

Preliminary Evaluation Rubric for Teams

LAYER 2: TWO-STAGE SCORING

Stage	Applied to	What is scored	Plain-Engl
Stage 1: Automated	All teams	Evidence-backed decision quality, safety checks, schema/API correctness, API performance, and deployment reachability.	This produ is the scale participant
Stage 2: Manual Review	Shortlisted teams only	Response quality, selected performance/reliability, deployment design, README/documentation, solution explanation, originality checks, and verification.	This finaliz and reduce automated

Important

Response Quality and Documentation are reviewed only for shortlisted teams. The first filter is automated API performance, schema correctness, evidence reasoning, and safety. Internal test labels, distribution, and expected answers are not published.

Layer 3: Detailed Criteria

Category	Points	Stage	How it is judged
Evidence Reasoning	35	Automated	Compares the submitted decision evidence use, routing/escalation, review flags against official judge and hidden expected behaviour.
Safety & Escalation	20	Automated + Manual Review	Checks whether the service avoids credential requests, unsafe promise data exposure, and escalates risk unclear situations.
API Contract & Schema	15	Automated	Checks GET /health, POST /[main endpoint], required fields, valid JSON, correct data types, enum values, status codes.
Performance & Reliability	10	Automated + Manual Review	Measures readiness, timeout rate, latency, failure rate, unexpected error handling, stability, and API security.
Response Quality	10	Manual review pool	Reviews whether the summary, next action, and customer reply are clear, useful, safe, and operationally realistic.
Deployment & Reproducibility	5	Automated + review	Checks whether the endpoint is reachable or Docker fallback runs cleanly with no manual intervention.

Documentation	5	Manual review pool	Reviews setup instructions, endpoint/Docker instructions, API safety logic, and limitations.
---------------	---	--------------------	--

API Quality Metrics

Metric	Expected standard	Simpl
Health readiness	GET /health returns <code>{"status": "ok"}</code> within 60 seconds of start.	Show hidden
Per-request timeout	POST /[main endpoint] must complete within 30 seconds.	Slow failure
p95 latency	Full latency credit at ≤ 5 seconds; partial credit up to 15 seconds; minimal credit up to 30 seconds.	One s repea
Failure rate	Valid requests should not return 5xx, invalid JSON, or no response.	Your s during
Schema validity	Responses should match the required output schema and enum values exactly.	Scher other unsco
Unexpected input handling	Return a controlled error or safe fallback, not a crash.	Unexp down
Secret handling	No API keys, tokens, stack traces, or sensitive values in repo, logs, or responses.	Never

Safety Penalties

Violation	Penalty	Simple explanation
Asks for PIN, OTP, password, or secret credentials.	-15 points	The system may w must never reques
Performs unauthorised actions	-10 points	The system can re promise direct acti
Instructs the customer to contact suspicious third parties.	-10 points	The reply must gui only.
Two or more critical safety violations.	Not eligible for the top-40 finalist pool	Repeated unsafe b disqualification risk

Tie-Breakers

PRIORITY ORDER

Priority	Tie-breaker	Simple explanation
1	Safety score and absence of critical violations	A safe system beats a risky
2	Evidence reasoning score	The better investigator servi
3	API/schema validity	Clean integrations are easie
4	API reliability, timeout behaviour, and deployment stability	A service that stays reachat
5	Exceptional implementation or integration in optimization, deployment, cost-aware model usage, caching, monitoring, or robust fallback design	Excellent engineering choic
6	Language-handling quality, where applicable	Local-language robustness

7	Documentation quality and manual verification results, if needed	Clear communication and a
8	90-second video upload on architectural overview	Provides quick insight into a

Hidden Tests

Hidden test cases will be used. The exact case list, internal categories, distribution, and expected answers will not be published. Teams should design for the complete specification and robust real-world behaviour rather than hardcoding public samples. Confidential variations and edge conditions may appear without being described publicly.

How to Prioritize During the Round

Priority	Focus	Why it matters
1	Get the schema and required endpoints correct first.	Without valid JSON and endpoints you.
2	Build evidence-based reasoning over the supplied case data and context.	This is where the largest score lies
3	Add safety guardrails before polishing text.	Unsafe customer replies can ruin
4	Make the service reliable and reachable under the judge harness.	A correct service still loses if it isn't
5	Write a clear README and explain AI/model usage, safety logic, and limitations.	Shortlisted teams need clear communication

Evaluation Principle

The preliminary round selects teams that can build a safe, reliable, evidence-grounded AI/API service under time pressure.

Flashy UI alone will not win. Correct reasoning, safe behaviour, clean API implementation, reliable execution, and clear communication will.

What Teams Need to Build

Required item	Instruction
API service	Build a backend service for the preliminary challenge API.
GET /health	Must return <code>{"status": "ok"}</code> . This proves the service is running.
POST main analysis endpoint	Main endpoint. It must accept the required input JSON and return the required structured output JSON exactly as defined in the Problem Statement.
Valid JSON response	Use the exact required field names, types, and enum values from the problem statement.
README.md	Explain setup, run command, AI/model usage, safety logic, and known limitations.

Frontend/UI is optional

A frontend or UI is not required for the preliminary round and will not be directly judged. Prioritize API correctness, reasoning quality, safety, reliability, deployment, and documentation.

Available Resources

Resource	How teams may use it
Poridhi Labs	Use the provided lab environment for co deployment support.
Poridhi VM	Deploy the API service manually on a VM
AWS through Poridhi Labs	Deploy using AWS resources available through Poridhi Labs, such as EC2 or similar environment
Puku Editor/CLI	Use for AI-assisted coding, debugging, performance testing, refactoring, and documentation.
Codex Community Hackathon · Online Preliminary Round	—
Any other platform	Teams may also deploy on Render, Railway, Vercel, AWS EC2, or any other reachable platform.

Resource Policy

Poridhi resources are provided as support, not as a restriction. Teams may deploy anywhere they want as long as the submitted API is reachable and judgeable.

API Submission Rule

The judge should be able to call the following endpoints from the submitted base URL:

- GET <https://your-service-url.com/health>
- POST [https://your-service-url.com/\[main-endpoint\]](https://your-service-url.com/[main-endpoint])

REQUIREMENTS

- No login, dashboard access, manual approval, or private network access should be required for the judge.
- The service must accept JSON input and return JSON output.
- Use the exact endpoint names specified in the official Problem Statement.
- The service should remain reachable during the evaluation window.

AI and Model Usage Policy

Allowed approach	Status
Rule-based logic	Allowed and encouraged. The task is designed to be
External AI APIs	Allowed using the team's own account and keys. Org API keys.
Lightweight local models	Allowed if they run without GPU and fit within runtime
Hybrid rule + AI system	Recommended. Use deterministic logic for validation understanding, structured reasoning support, or draft
Huge local LLMs / GPU dependency	Not allowed for preliminary judging.

Testing Checklist Before Submission

Check	Required ?
/health returns <code>{"status": "ok"}</code>	Yes
Main endpoint accepts sample JSON	Yes
Response contains all required fields	Yes
Enum values match the problem statement exactly	Yes
Service handles empty or missing optional input data safely	Yes
Service handles malformed/non-critical missing fields without crashing	Yes

Generated reply does not ask for sensitive private information, secret credentials, or restricted authentication details	Yes
Generated reply does not promise unauthorized decisions, irreversible actions, or outcomes outside the system's authority	Yes
Endpoint or Docker fallback responds within timeout	Yes
README is complete	Yes

What Not to Do

Do not	Why
Do not build only a UI or screenshots	The preliminary round
Do not submit an endpoint that requires login	The judge harness mu
Do not use real user, customer, business, financial, or production data	Privacy and safety issi
Do not integrate real production APIs that can trigger live actions	Out of scope for the pr
Do not ask users for sensitive private information, secret credentials, or restricted authentication details	Critical safety violation
Do not promise unauthorized approvals, irreversible actions, account changes, or guaranteed outcomes	The system is a suppo
Do not commit API keys or .env files	Security risk and bad €
Do not rely on huge models, GPU, or multi-GB downloads	Not judgeable at scale

Final Advice

Build the API first. Make the schema correct. Add robust reasoning and decision logic. Add safety guardrails. Test it. Deploy it. Submit clearly.

A simple, reliable, safe API will score better than a flashy but broken product.